

Sign Language Studio — Video Processing Pipeline

DRS automation system · /Users/signlab/drs · orchestrated by **startupScript.py** (12 services, auto-restart, scheduled restarts 06:00 & 20:00)
· generated 2026-06-10

☐ Capture ☐ Storage / transfer ☐ Processing ☐ Upload to signcollect.nl ☐ Parallel branch

A — CAPTURE

Sony cameras (L / M / R / A / B)

Multi-angle recording; RemoteCli + Node server download files from the cameras

/Volumes/cacheDisk/signCollect/studioFiles/<date>/raw/*.MP4

▼ *rsync, every 15 min, waits 5 min after downloads finish (moveFiles.py)*

B — CENTRAL STORAGE

rclone cloud mount

Remote **signcollect**: mounted locally; all later stages read & write here

/Users/signlab/signCollect/.../studioFiles/<date>/raw/

▼ *batch_queue.py copies ≤50 files to local import/ and drives DaVinci Resolve*

C — PROFESSIONAL EDIT

DaVinci Resolve render (batch_queue.py)

Timeline per video, Fusion comp (green-screen removal, background), render to export/

import/ → export/ → .../<date>/post_noncropped/

▼ *crop.py picks up rendered videos*

D — AI CROPPING

MediaPipe pose crop (crop.py)

Detects signer pose, auto-rotates tilted footage, crops 1440x1252 around the signer, re-encodes (h264), makes thumbnail

.../<date>/post_cropped/ + thumbnails/

▼ *HTTP upload*

E — PUBLISH

signcollect.nl

Cropped video + thumbnail posted to **videoProc/upload_post.php**

— parallel branches reading from the same storage —

BRANCH 1 — CONVERTFILES.PY

H.264 web conversion

raw/ → ffmpeg (libx264 + aac) → converted/

+ thumbnail → upload to **upload2.php**. Hourly.

BRANCH 2 — QR_SCANNER_SERVICE.PY

QR detection

Scans thumbnails/ (last 7 days) with QReader + pyzbar; results to **qr/qrResultReceiver.php**. Every 15 min.

BRANCH 3 — LISTFILES.PY

Inventory & coverage report

Counts files per camera, checks L/M/R completeness + glosld coverage; posts to **listFiles.php**. Hourly.

Pipeline services — what each one does

Service	Purpose	Reads from	Writes / sends to	Schedule
moveFiles.py	Moves freshly downloaded camera files from the local cache disk into the cloud mount, organized into date folders parsed from the filename (e.g. R20250527_1851.MP4).	/Volumes/cacheDisk/.../<date>/raw/ ; WebSocket (port 8081) download events	signCollect mount .../<date>/raw/ via rsync	Every 15 min
batch_queue.py	Drives DaVinci Resolve: builds a timeline per raw video, applies the Fusion composition (config/Settings.setting) for green-screen removal and background replacement, renders in batches of ≤50. Writes .skip markers for failed files.	mount .../raw/ → local import/	local export/ → mount .../post_noncropped/	Continuous
crop.py	AI cropping: extracts frames, runs MediaPipe pose detection to find the signer (head-to-waist bounds, nose-centered), corrects camera tilt, crops to 1440x1252, re-encodes with ffmpeg, generates a thumbnail, and uploads the result.	mount .../post_noncropped/ ; temp frames in temp/frames_* (auto-cleanup >1 h / >10 GB)	mount .../post_cropped/ + thumbnails/ ; upload → upload_post.php	Continuous
convertFiles.py	Web-friendly conversion of the raw recordings: ffmpeg libx264/aac/yuv420p baseline profile + a thumbnail taken at the 50% mark, then upload.	mount .../raw/ (skips dates that already have converted/)	mount .../converted/ + thumbnails/ ; upload → upload2.php	Hourly
qr_scanner_service.py	Scans thumbnails for QR codes (used to link recordings to session/sign IDs) with a 4-worker pool (QRReader + pyzbar). Remembers what it already scanned.	mount .../thumbnails/*.jpg (last 7 days); state in qr/scanned_files.json	upload → qr/qrcodeReceiver.php	Every 15 min
listFiles.py	Quality/completeness reporting: counts files per camera (L/M/R/A/B) for the last 10 dates, verifies all cameras recorded equally, computes glosid coverage against the expected list from CR.php .	mount .../raw/ + sidecar JSONs; cache in logs/glosids_cache/	POST inventory JSON → listFiles.php	Hourly

How the stages chain: each stage is decoupled — services don't call each other. They watch directories on the shared rclone mount and act when new files appear: raw/ → (Resolve) → post_noncropped/ → (MediaPipe) → post_cropped/ . That makes the pipeline self-healing: if a stage was down, it simply catches up on the backlog when restarted.

Infrastructure & support services

Service	Purpose	Notes
startupScript.py	Orchestrator: starts all 12 services, monitors them every 5 s, auto-restarts crashes (max 5 restarts / 5 min, then 15-min cooldown), checks rclone mount health every 60 s, full restart of everything daily at 06:00 and 20:00.	Logs to startup.log + per-service logs/*.log (10 MB rotation)
watchdog.sh	Watches the orchestrator itself via its PID file and relaunches it within 30 s if it dies — the last line of defense.	startup.pid , watchdog.log
rclone mount	Mounts the remote signcollect: storage at /Users/signlab/signCollect with full VFS caching (up to 4 TB on /Volumes/cacheDisk/rclone , 5 s write-back). All pipeline stages read/write through this mount.	Health-checked via marker file; auto-remounted on failure
startServer_beta.js	Express + WebSocket server for camera communication: notifies clients when camera downloads complete (fileDownloaded events that moveFiles.py listens for) and streams/serves video files for preview.	HTTP :8080, WebSocket :8081
network_manager.py	Keeps the studio Mac reachable: tests internet connectivity every 30 s, restarts ethernet (en0) on loss, restarts Tailscale after boot.	Required for rclone + uploads
mouse.py	Moves the mouse slightly every 60 s to prevent macOS screen lock / sleep, which would interrupt DaVinci Resolve rendering.	pyautogui
keyboard_monitor.py	Tracks real keyboard activity (ignoring the simulated mouse) so the system can tell an actual operator apart from automation.	Writes last_keyboard_activity.txt

Key directories

Path	Role
/Volumes/cacheDisk/.../studioFiles/	Camera download landing zone (local disk)
/Users/signlab/signCollect/.../studioFiles/	Cloud mount — central store, one folder per date
<date>/raw/	Original camera files + sidecar JSON
<date>/post_noncropped/	After DaVinci Resolve render
<date>/post_cropped/	Final AI-cropped videos
<date>/converted/ + thumbnail s/	Web H.264 copies + preview images
drs/import/ · drs/export/	Local Resolve staging in/out
drs/temp/	MediaPipe frame extraction (auto-cleaned)
drs/logs/	Per-service logs, 10 MB rotation

External integrations

Endpoint / tool	Used for
signcollect.nl/videoProc/upload_post.php	Final cropped video + thumbnail upload (crop.py)
signcollect.nl/videoProc/upload2.php	Converted H.264 + thumbnail upload (convertFiles.py)
signcollect.nl/qr/qrResultReceiver.php	QR scan results (qr_scanner_service.py)
signcollect.nl/listFiles.php	File inventory & coverage report (listFiles.py)
signcollect.nl/CR.php	Expected glosId list per date (listFiles.py)
SignCollect monitor API	Heartbeats from every service (uptime dashboard)
DaVinci Resolve API	Timeline creation, Fusion comps, rendering
MediaPipe / OpenCV / ffmpeg	Pose detection, cropping, encoding
rclone (signcollect: remote)	Cloud storage mount